UD3 CSS

POSICIONAMIENTO

CFGS DAM

Módulo Lenguajes de marcas

LMSG

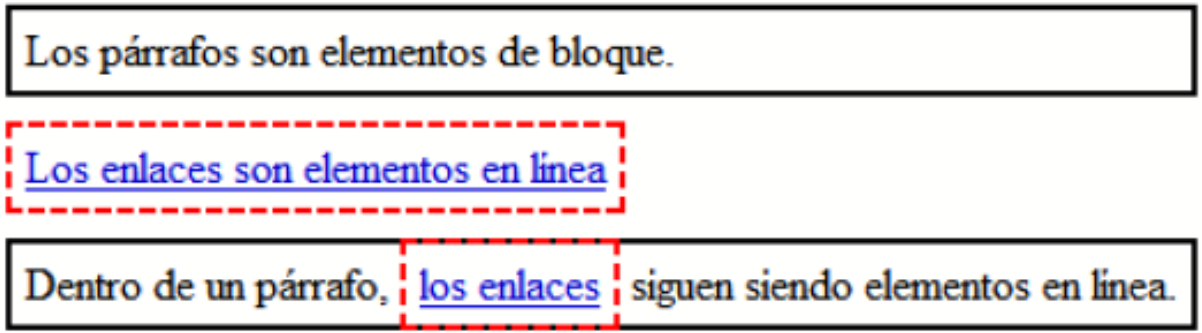
I.E.S. Serra Perenxisa

Introducción

- Una de las grandes e históricas peticiones de los diseñadores web ha sido la posibilidad de poder colocar contenido en una posición concreta de la página, haciendo que los contenidos no se vayan mostrando en el orden en el que aparecen en el código, sino en el que decidimos libremente.
- Fue la principal aportación de CSS2 y ha provocado que, en lugar de utilizar tablas para definir la maquetación de la página, ahora sean las capas las encargadas de hacerlo. Las capas son elementos que permiten una distribución de contenidos más libre. La etiqueta div suele ser la encargada de crear capas, aunque con HTML5 son otros elementos los que también se usan para esta tarea como section o artículo , ya que div no es una etiqueta semántica (no dice qué tipo de contenido posee, si es un artículo, un título, un menú,…).
- Hay que tener en cuenta que la posición de un elemento también estará influido por el valor de la propiedad display, como se explicó en el punto anterior.

Recordemos los elementos de bloque y de línea

- **De bloque:** empiezan en una nueva línea y ocupan todo el espacio disponible hasta el final de la línea.
- **En línea:** no empiezan necesariamente en nueva línea y sólo ocupan el espacio necesario para mostrar sus contenidos.



Posicionamiento

- Los navegadores crean y posicionan de forma automática todas las cajas. La propiedad posicionamiento (*position*) sirve para indicar cómo un elemento fluye por la página
- Con esta propiedad podremos indicar si el elemento al que realmente se aplica, se comportará como una capa de contenidos que se colocará libremente en la página.

Tipos de posicionamiento

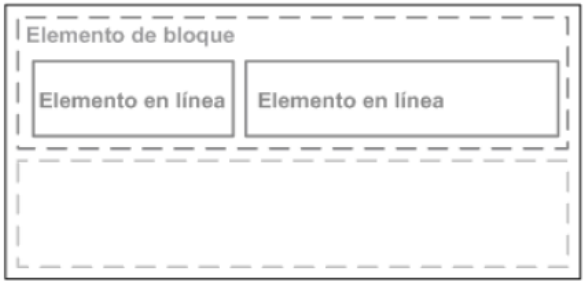
- **Estático:** posiciona elementos en función del flujo natural (defecto).
- **Relativo:** posiciona un elemento desplazándolo respecto de su posición natural.
- **Fijo:** posiciona el elemento de manera relativa al *viewport*.
- **Absoluto:** posiciona el elemento relativo a su padre.
- **Sticky**: posiciona el elemento entre relativo y fijo en función del *scroll*.

Valores para *position*

Static

Se establece por defecto (sin especificar nada) o *position:static*

- Se aplica por defecto y aparece en la posición que dicta el flujo natural del código (de izquierda a derecha y de arriba abajo). Ignorará las propiedades de posicionamiento (left, right, top, bottom). Sólo se tiene en cuenta si el elemento es de bloque o en línea.



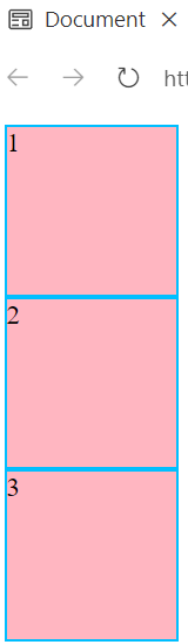
Valores para *position*

Static o posicionamiento normal. Ejemplo

```css
.cuadro{
    width: 100px;
    height: 100px;
    background-color: lightpink;
    border: 2px deepskyblue solid;
}
.movido{
    left: 100px;
}
```

*Da igual que ponga **left**, ninguna caja se desplaza respecto de su posición original*

```html
<div class="cuadro">1</div>
<div class="cuadro movido">2</div>
<div class="cuadro">3</div>
```



Valores para *position*

Relative

Se establece con la propiedad *position:relative*

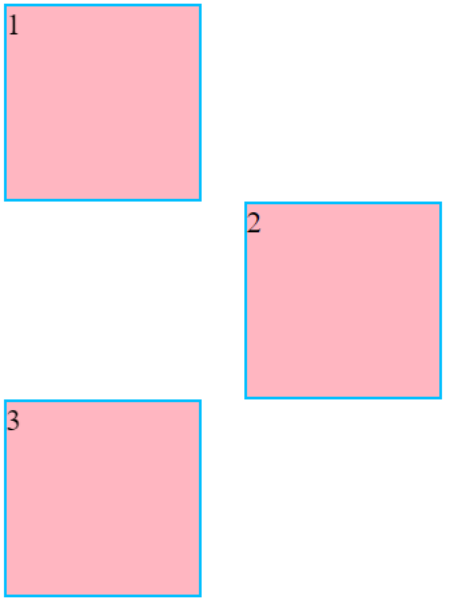
- Como static, pero sí permite modificar su posición inicial usando left, top, right y bottom.
- En modo relative , el elemento marcado con este posicionamiento sí se mueve con las propiedades de posicionamiento, pero deja el **vacío** que le correspondería en modo static .
- En definitiva, **relative provoca un desplazamiento al elemento.**

Valores para *position*

Relative. Ejemplo

```css
.cuadro{
    width: 100px;
    height: 100px;
    background-color: lightpink;
    border: 2px deepskyblue solid;
}
.movido{
    position: relative;
    left: 125px;
}
```

```html
<div class="cuadro">1</div>
<div class="cuadro movido">2</div>
<div class="cuadro">3</div>
```



Valores para *position*

Fixed

Se establece con la propiedad *position:fixed*

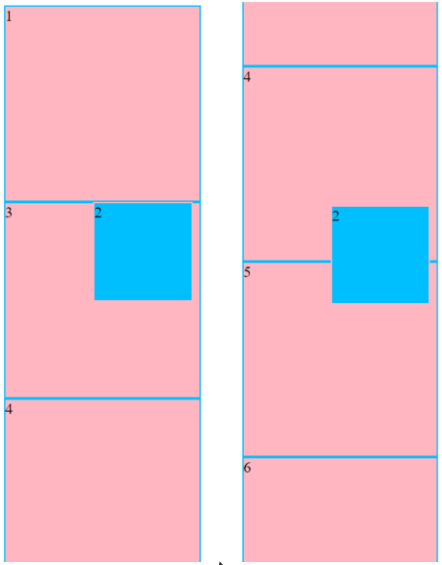
- El elemento flota libremente por la ventana y abandona el vacío que, por su orden natural le correspondería. De hecho, este vacío se elimina y lo ocupa el siguiente elemento disponible. La posición del elemento se fija en base a las coordenadas de la ventana, independientemente de donde esté el elemento situado o contenido. Es decir, toma como referencia la ventana del documento, de forma que la esquina superior izquierda será la coordenada 0,0.
- La posición de los elementos que están con fixed son inamovibles dentro de la ventana del navegador, no modifican su posición ni aunque el usuario suba o baje la página en la ventana con el scroll.

Valores para *position*

Fixed. Ejemplo

```css
.cuadro{
    width: 200px;
    height: 200px;
    background-color: lightpink;
    border: 2px deepskyblue solid;
}
.movido{
    position: fixed;
    left: 100px;
    width: 100px;
    height: 100px;
    background-color: deepskyblue;
    border: 2px lightpink solid;
}
```

```html
<div class="cuadro">1</div>
<div class="cuadro movido">2</div>
<div class="cuadro">3</div>
<div class="cuadro">4</div>
<div class="cuadro">5</div>
<div class="cuadro">6</div>
```



Vas desplazándote y el cuadro 2 se mantiene en la misma posición

Valores para *position*

Absolute

Se establece con la propiedad *position:absolute*

- La **caja posicionada sale de su posición normal** y el resto de los elementos ignoran la nueva posición del elemento, pudiéndose producir solapamientos de cajas.
- Sus coordenadas (top, left, right, bottom) se calculan respecto al primer contenedor que tenga un posicionamiento explícito (como relative, absolute o fixed).
- Recordemos que con **fixed** la posición se establece respecto de la ventana, mientras que con **absolute** es respecto de su contenedor,

Valores para *position*

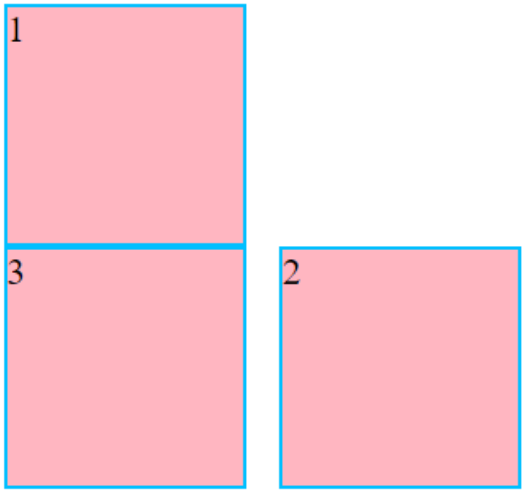
Posicionamiento absoluto. Ejemplo I

```css
<style>
    .cuadro {
        width: 100px;
        height: 100px;
        background-color: lightpink;
        border: 2px deepskyblue solid;
    }
    .movido {
        position: absolute;
        left: 125px;
    }
</style>
```

El dos se posiciona fuera del flujo normal del documento

```html
<div class="cuadro">1</div>
<div class="cuadro movido">2</div>
<div class="cuadro">3</div>
```

Ambos divs siguen el flujo natural del documento, primero 1 y luego 3



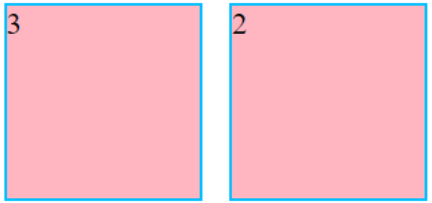
Valores para *position*

Posicionamiento absoluto. Ejemplo II

```
<style>
    .cuadro {
        width: 100px;
        height: 100px;
        background-color: lightpink;
        border: 2px deepskyblue solid;
        position: absolute;
    }
    .movido {
        position: absolute;
        left: 125px;
    }
</style>
```

Ahora todos son absolutos

```
<div class="cuadro">1</div>
<div class="cuadro movido">2</div>
<div class="cuadro">3</div>
```



cuadro3 encima de cuadro1 y el cuadro 2 movido a 125px a la izquierda con relación al body

Valores para *position*

Posicionamiento sticky.

Se establece con la propiedad *position:sticky*

- El elemento **cambia entre position:relative y position:fixed** en función del *scroll*. Es decir, el elemento se posiciona de forma relativa hasta que un cierto *scroll* llega al mismo y en ese momento se fija "sticks" y queda fijo.
- Es muy habitual en menús superiores.
- Siempre debe especificarse **al menos una propiedad top, right, bottom o left**.

Posicionamiento

¿Qué pasa si no hay un contenedor con posición explícita?

- Si el elemento está dentro de un contenedor cuyo position es static (que es el valor predeterminado), el navegador sigue buscando hacia arriba en el árbol de elementos (ancestros o "padres", "abuelos") hasta encontrar uno cuyo position no sea static. Si no encuentra ningún ancestro con posicionamiento explícito, usará el body o el borde de la ventana como referencia.

Posicionamiento

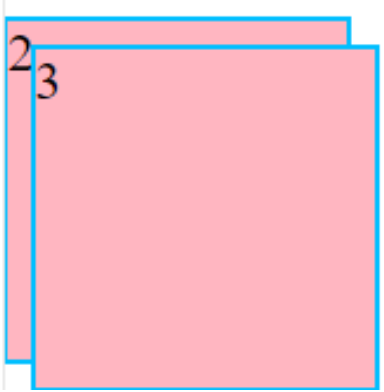
Más ejemplos

```
<style>
    .cuadro {
        width: 100px;
        height: 100px;
        background-color: lightpink;
        border: 2px deepskyblue solid;
        position: absolute;
    }

    .movido {
        position: absolute;
        left: 0px;
        top: 0px;
    }
</style>
```

Posición 0,0 respecto a su padre (body)

```
<div class="cuadro">1</div>
<div class="cuadro movido">2</div>
<div class="cuadro">3</div>
```



cuadro3 encima de cuadro1 y el cuadro 2 movido con relación a la posición 0,0 respecto al body que es su contenedor

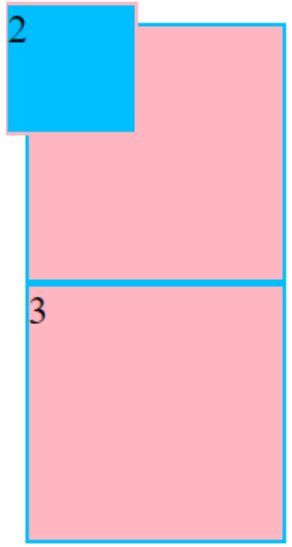
Posicionamiento

Padre static, hijo absolute

```css
.cuadro{
    width: 100px;
    height: 100px;
    background-color: lightpink;
    border: 2px deepskyblue solid;
}
.movido{  /*hijo*/
    position: absolute;
    left: 0px;
    top: 0px;
    width: 50px;
    height: 50px;
    background-color: deepskyblue;
    border: 2px lightpink solid;
}
```

Posición 0,0 respecto al body

```html
<div class="cuadro">1
    <div class="cuadro movido">2</div>
</div>
<div class="cuadro">3</div>
```



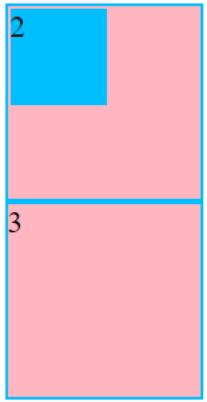
Cuadro 1 y dentro cuadro 2 desplazado respecto a la posición de su abuelo (body) ya que su padre tiene posición static, seguido de cuadro 3.

Posicionamiento

Padre relative, hijo absolute

```css
.cuadro{
    width: 100px;
    height: 100px;
    background-color: lightpink;
    border: 2px deepskyblue solid;
}
.padre{
    position: relative;
}
.movido{  /*hijo*/
    position: absolute;
    left: 0px;
    top: 0px;
    width: 50px;
    height: 50px;
    background-color: deepskyblue;
    border: 2px lightpink solid;
}
```

Posición 0,0 respecto a su padre cuadro 1



Cuadro 1 y dentro cuadro 2 desplazado respecto a la posición de su padre (relative)

```html
<div class="cuadro padre">1
    <div class="movido">2</div>
</div>
<div class="cuadro">3</div>
```

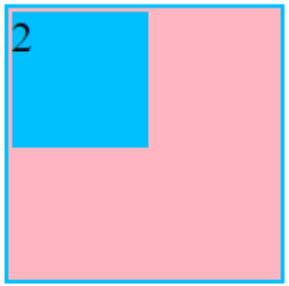
Posicionamiento

Padre absolute, hijo absolute

```css
.cuadro{
    width: 100px;
    height: 100px;
    background-color: lightpink;
    border: 2px deepskyblue solid;
    position: absolute;
}
.movido{  /*hijo*/
    position: absolute;
    left: 0px;
    top: 0px;
    width: 50px;
    height: 50px;
    background-color: deepskyblue;
    border: 2px lightpink solid;
}
```

Posición 0,0 respecto a su padre cuadro 1

```html
<div class="cuadro">1
    <div class="cuadro movido">2</div>
</div>
```



Cuadro 1 y dentro cuadro 2 desplazado respecto a la posición de su padre (absolute)

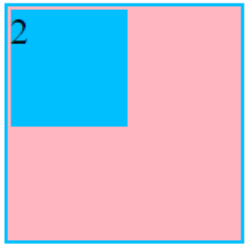
Posicionamiento

Padre absolute, hijo absolute

```
.cuadro{
    width: 100px;
    height: 100px;
    background-color: lightpink;
    border: 2px deepskyblue solid;
}
.padre{
    position: absolute;
}
.movido{  /*hijo*/
    position: absolute;
    left: 0px;
    top: 0px;
    width: 50px;
    height: 50px;
    background-color: deepskyblue;
    border: 2px lightpink solid;
}
```

Posición 0,0 respecto a su padre cuadro 1

Añadimos cuadro 3 static



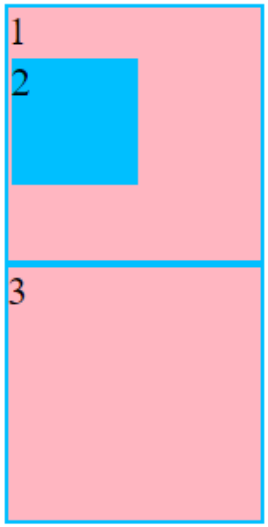
Orden: Cuadro 3 que es el que tiene el flujo normal. Luego Cuadro 1 y dentro cuadro 2 desplazado respecto a la posición de su padre (absolute)

```
<div class="cuadro padre">1
    <div class="movido">2</div>
</div>
<div class="cuadro">3</div>
```

Posicionamiento

Padre relative, hijo relative.

```css
.cuadro{   /*static*/
    width: 100px;
    height: 100px;
    background-color: lightpink;
    border: 2px deepskyblue solid;
}
.movido{   /*hijo*/
    position: relative;
    left: 0px;
    top: 0px;
    width: 50px;
    height: 50px;
    background-color: deepskyblue;
    border: 2px lightpink solid;
}
.relativo{  /*padre*/
    position:relative;
}
```



No se solapa, comienza después del contenido del cuadro 1

```html
<div class="cuadro relativo">1 <!--padre-->
    <div class="movido">2</div>
</div>
<div class="cuadro">3</div>
```

Establecer coordenadas de posicionamiento

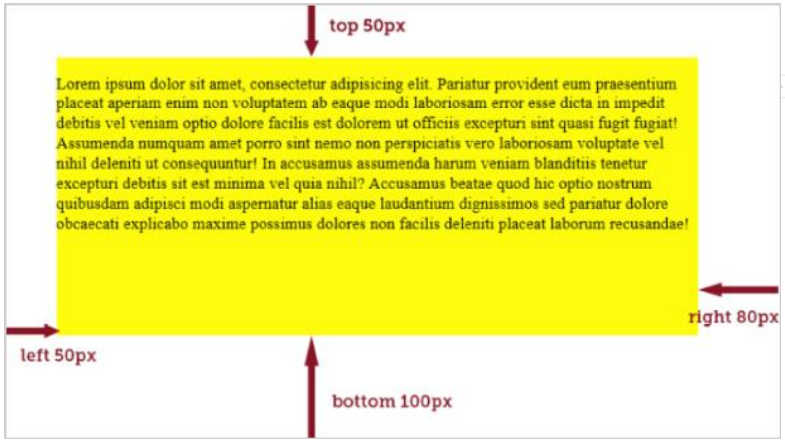
- Cuatro propiedades se encargan de mostrar un elemento en una posición exacta en la página
  - left *permite indicar la coordenada izquierda del elemento. Tomada desde el margen izquierdo*
  - top *Coordenada superior. Toma desde el borde superior.*
  - bottom *Coordenada inferior. Distancia en el borde inferior*
  - right *Coordenada derecha. Distancia en el borde derecho.*
- Las coordenadas se calculan desde la referencia de coordenadas establecidas por la propiedad position . Si es fixed se toman los bordes de la ventana. Es incompatible usar bottom y right y al mismo tiempo, los tamaños con width y height .

Establecer coordenadas de posicionamiento

Ejemplo

```css
div{
    position: fixed;
    left: 50px;
    top: 50px;
    right: 80px;
    bottom :100px;
    background-color: yellow;
}
```

```html
<div>
    <p> Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest dolorem ut officiis excepturi sint quasi fugit fugiat! Assumenda numquam amet porro sint nemo non perspiciatis veure laboriosam voluptate vel nihil deleniti ut consequuntur! In accusamus assumenda harum veniam blanditiis tenetur excepturi debitis sit est minima vel quia nihil? Accusamus beatae quod hic
    optio nostrum quibusdam adipisci modi aspernatur àlies eaque laudantium dignissims set pariatur dolore obcaecati explicabo maxime possimus dolors non façalis deleniti placeat laborum recusandae !</p>
</div>
```

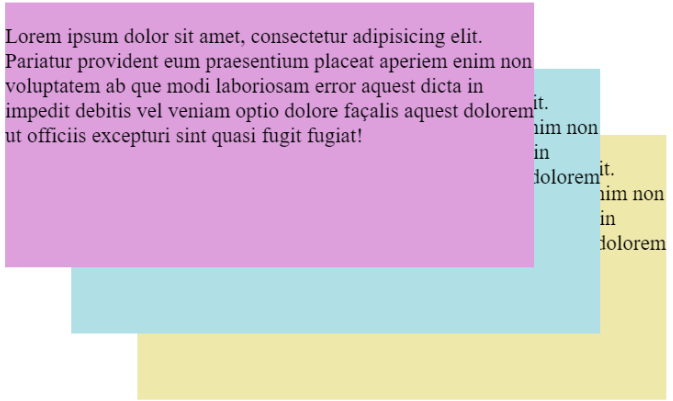


Z-index (Coordenada z)

- Permite controlar la posición tridimensional de las cajas posicionadas.
- Se controla mediante la propiedad *z-index*, pudiendo crear páginas con varios niveles o capas.
- Realmente, lo que aporta es la posibilidad de que los elementos se solapen y podamos decidir qué elemento está por encima del otro. Quien tenga la coordenada z más alta quedará sobre los demás elementos.
- El texto normal (de elementos que no tengan marcado un *z-index*) está en el nivel cero; con lo cual, si indicamos un nivel negativo en la propiedad *z-index* de un elemento, entenderemos que detrás del contenido normal (no se verá el elemento, quedará tapado).

Z-index Ejemplo

```css
.capa1 {
  position: fixed;
  left: 50px;
  top: 50px;
  width: 400px;
  height: 200px;
  background-color: plum;
  z-index: 1;
}
.capa2 {
  position: fixed;
  left: 100px;
  top: 100px;
  width: 400px;
  height: 200px;
  background-color: powderblue;
}
.capa3 {
  position: fixed;
  left: 150px;
  top: 150px;
  width: 400px;
  height: 200px;
  background-color: palegoldenrod;
  z-index: -1;
}
```




```html
<div class="capa1">
  <p> Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest dolorem ut officiis excepturi sint quasi fugit fugiat!</p>
</div>
<div class="capa2">
  <p> Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest dolorem ut officiis excepturi sint quasi fugit fugiat!</p>
</div>
<div class="capa3">
  <p> Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest dolorem ut officiis excepturi sint quasi fugit fugiat!</p>
</div>
```

Posicionamiento flotante (*float*)

- Normalmente los elementos no flotan. Cuando colocamos, por ejemplo, una imagen, el contenido que va a continuación aparece en su línea base, dejando un vacío poco estético. Podemos hacer que este contenido contornee la imagen a su derecha (float:left) o a su izquierda (float:left).
- Con este posicionamiento, una caja se hace flotante, desplazándose lo más a la izquierda o a la derecha de la posición original.
- La caja deja de pertenecer al flujo natural de la página, el resto de cajas fluyen alrededor de la caja flotante, si no caben, pasan a la siguiente línea.

```
float: left;
```

Float

- Los posibles valores de la propiedad float son:
  - left *El elemento flota a la izquierda. El contenido inmediatamente siguiente lo contornea a la derecha, siempre que quepa.*
  - right *El elemento flota a la derecha.*
  - none *Anula el posicionamiento flotante. Valor por defecto.*

- Esta propiedad está pensada para elementos de bloque, que son los que tienen un *display* con valor **block** o **inline-block** .

```
float: left;
```

Float. Ejemplo

```css
img{
width: 150px;}
```

```css
img{
float: none;
width: 150px;}
```



Lorem ipsum dolor sit amet consectetur adipisicing elit. Asperiores, nobis in, accusantium autem repudiandae ratione pariatur tenetur reiciendis placeat expedita dolore magnam assumenda hic beatae maiores eum sit incidunt itaque.

```html
<p>
  <img src="./gatito.jpg" alt="gato">Lorem ipsum dolor sit amet consectetur adipisicing elit. Asperiores, nobis in, accusantium autem repudiandae ratione pariatur tenetur reiciendis placeat expedita dolore magnam assumenda hic beatae maiores eum sit incidunt itaque.</p>
```

Float. Ejemplo

```css
img{
float: left;
width: 150px;}
```



Lorem ipsum dolor sit amet consectetur adipisicing elit. Asperiores, nobis in, accusantium autem repudiandae ratione pariatur tenetur reiciendis placeat expedita dolore magnam assumenda hic beatae maiores eum sit incidunt itaque.

```css
img{
float: right;
width: 150px;}
```

Lorem ipsum dolor sit amet consectetur adipisicing elit. Asperiores, nobis in, accusantium autem repudiandae ratione pariatur tenetur reiciendis placeat expedita dolore magnam assumenda hic beatae maiores eum sit incidunt itaque.



```html
<p>
  <img src="./gatito.jpg"
  alt="gato"> Lorem ipsum
  dolor sit amet consectetur
  adipisicing elit.
  Asperiores, nobis in,
  accusantium autem
  repudiandae ratione
  pariatur tenetur reiciendis
  placeat expedita dolore
  magnam assumenda hic beatae
  maiores eum sit incidunt
  itaque.</p>
```

Float. Más ejemplos - Fluyendo a la derecha

```
p {
  width: 300px;
  border: 1px solid black;
  background-color: yellow;
  }
p.flotante_d{
            float: right;
}
```

párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2 párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2 párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2

párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1 párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1 párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1

párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3 párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3 párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3

El párrafo 1 se desplaza completamente a la derecha, el resto de párrafos ocupan su lugar original

```
<p class="flotante_d">párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1
párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1
párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1</p>
<p>párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2
párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2
párrafo 2párrafo 2párrafo 2párrafo 2párrafo 2</p>
<p>párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3
párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3
párrafo 3párrafo 3párrafo 3párrafo 3párrafo 3</p>
```

Float. Más ejemplos - Fluyendo a la izquierda

```
p {
  width: 200px;
  border: 1px solid black;
  background-color: yellow;
  }
p.flotante_i{
            float: left;
}
```

párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1 párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1párrafo 1

Float. Más ejemplos - Todas flotantes

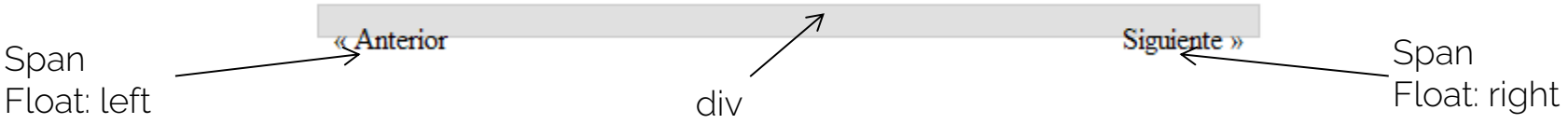
```css
p {
  width: 200px;
  border: 1px solid black;
  background-color: yellow;
  }
p.flotante_i{
            float: left;
}
```

párrafo 1párrafo 1párrafo 1

Clear

- Para “romper” el efecto de los float.
- Permite forzar a un elemento a mostrarse debajo de cualquier caja flotante.
- La propiedad indica el lado del elemento que no debe ser adyacente a ninguna caja posicionada de forma flotante. Valores posibles:
  - **left**, el elemento se desplaza de forma descendente hasta que pueda colocarse en una línea en la que no haya ninguna caja flotante en el lado izquierdo.
  - **right**, igual para el derecho.
  - **both**: despeja los lados izquierdo y derecho del elemento

Algunos problemas provocados por *float*




```
div#paginacion {
  border: 1px solid #CCC;
  background-color: #EOEOEO;
  padding: .5em;
}
.derecha    { float: right; }
.izquierda { float: left;  }
-->
</style>
</head>
<body>
<div id="paginacion">
  <span class="izquierda">&laquo; Anterior</span>
  <span class="derecha">Siguiente &raquo;</span>
</div>
```

- *Como los dos elementos creados dentro del elemento se han posicionado mediante float, han salido del flujo normal del documento.*
- *Así, el elemento no tiene contenidos y por eso no llega a cubrir el texto de los dos elementos *

Solución 1: limpiar el *float* con *clear*

- Creamos un elemento sin contenido dentro de div con clear: both para limpiar el float.

```
div#paginacion {
  border: 1px solid #CCC;
  background-color: #EOEOEO;
  padding: .5em;
}
.derecha    { float: right; }
.izquierda { float: left;  }
div.clear   { clear: both; }

-->
</style>
</head>
<body>
<div id="paginacion">
  <span class="izquierda">&laquo; Anterior</span>
  <span class="derecha">Siguiente &raquo;</span>
  <div class="clear"></div>
</div>
```

« Anterior Siguiente »

Solución 2: Utilizando *overflow*

- Dar una altura muy pequeña a la división y establecer la propiedad: *overflow*: hidden.

```
div#paginacion {
  border: 1px solid #CCC;
  background-color: #EOEOEO;
  padding: .5em;
  height: 1%;
  overflow: hidden;
}
.derecha    { float: right; }
.izquierda { float: left;  }

-->
</style>
</head>
<body>
<div id="paginacion">
  <span class="izquierda">&laquo; Anterior</span>
  <span class="derecha">Siguiente &raquo;</span>
</div>
```

Lo vemos enseguida …

« Anterior Siguiente »

Visualización

- Para controlar la visualización de los elementos existen entre otras las siguientes propiedades:
  - ***Overflow*** para tratar el desbordamiento.
  - ***Visibility*** para mostrar o esconder elementos pero que todavía forman parte de los contenidos.
  - ***Display*** para controlar cómo se visualiza un elemento. La ocultación es de forma absoluta ya que deja de formar parte de los contenidos y no sería accesible desde lenguajes como JavaScript. El resto de elementos de la página se mueven a ocupar su lugar. Esta propiedad ya la vimos en el tema 10. Las opciones de maquetación avanzada las veremos más adelante.

Visualización

Overflow (desbordamiento)

- Es una propiedad que permite decidir la política que debe adoptar el navegador cuando un contenido supera el tamaño previsto del contenedor.
- Valores:
  - visible *el contenido se muestra aunque supere el tamaño previsto.*
  - hidden *oculta el contenido que sobrepasa el tamaño prefijado para la capa.*
  - scroll *se muestran barras de desplazamiento para poder acceder al contenido que supera el tamaño establecido para la capa.*
  - auto *igual que el anterior, pero las barras sólo se muestran si el contenido no cabe al tamaño fijado para la capa; si es posible, no se muestra ninguna barra de desplazamiento.*
- overflow-x overflow-y para tratar el desbordamiento horizontal y vertical de forma distinta.

Visualización

Overflow - Ejemplo

```css
.peque {
  position: fixed;
  left: 50px;
  top: 50px;
  width: 400px;
  height: 100px;
  background-color: plum;
  overflow:visible
}
```

Es lo mismo que si no ponemos nada

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest dolorem ut officiis excepturi sint quasi fugit fugiat!

```html
<div class="peque">
  <p> Lorem ipsum dolor sit amet,
  consectetur adipisicing elit.
  Pariatur provident eum praesentium
  placeat aperiem enim non voluptatem
  ab que modi laboriosam error aquest
  dicta in impedit debitis vel veniam
  optio dolore façalis aquest dolorem
  ut officiis excepturi sint quasi
  fugit fugiat!</p>
</div>
```

Visualización

Overflow - Ejemplo

```
overflow:visible;
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest dolorem ut officiis excepturi sint quasi fugit fugiat!

```
overflow:hidden;
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest dolorem

```
overflow:scroll;
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest

```
overflow:auto;
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Pariatur provident eum praesentium placeat aperiem enim non voluptatem ab que modi laboriosam error aquest dicta in impedit debitis vel veniam optio dolore façalis aquest

Visualización
Visibility

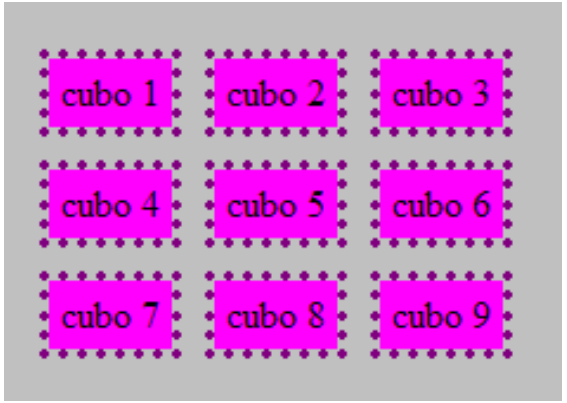
- Permite hacer invisible un elemento, haciendo que el navegador cree la caja sin mostrarla. El resto de elementos no modifican su posición, ya que la caja no se ve, pero sigue ocupando sitio.
- Valores:
  - visible *valor por defecto que establece que el elemento es visible en la página*
  - hidden *el elemento es invisible, pero ocupa el espacio que le correspondería.*
  - collapse *sólo se puede utilizar en las filas, grupos de filas, columnas y grupos de columnas de una tabla. Su efecto es ocultar completamente la fila y/o columna, mostrando otros contenidos en ese lugar.*

Visualización

Visibility - Ejemplo

```css
body {background-color: #c0c0c0; }
p {
  float: left;
  padding: 5px;
  margin: 5px;
  border: 4px dotted #800080;
  background-color: #ff00ff;
  }
div { clear: both; }
```

```html
<div>
     <p>cubo 1</p>
     <p>cubo 2</p>
     <p>cubo 3</p>
</div>
<div>
     <p>cubo 4</p>
     <p>cubo 5</p>
     <p>cubo 6</p>
</div>
<div>
     <p>cubo 7</p>
     <p>cubo 8</p>
     <p>cubo 9</p>
</div>
```

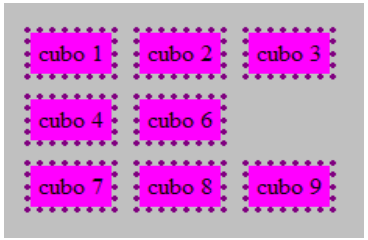


Visualización

Ejemplo – *display* vs *visibility*

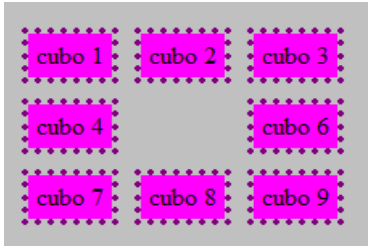
```
body {background-color: #c0c0c0; }
p {
  float: left;
  padding: 5px;
  margin: 5px;
  border: 4px dotted #800080;
  background-color: #ff00ff;
  }
div { clear: both; }
p.cinco {
    display: none;
}
```

```
<div>
      <p>cubo 1</p>
      <p>cubo 2</p>
      <p>cubo 3</p>
</div>
<div>
      <p>cubo 4</p>
      <p class="cinco">cubo 5</p>
      <p>cubo 6</p>
</div>
<div>
      <p>cubo 7</p>
      <p>cubo 8</p>
      <p>cubo 9</p>
</div>
```




```
body {background-color: #c0c0c0; }
p {
  float: left;
  padding: 5px;
  margin: 5px;
  border: 4px dotted #800080;
  background-color: #ff00ff;
  }
div { clear: both; }
p.cinco {
    visibility: hidden;
}
```

```
<div>
      <p>cubo 1</p>
      <p>cubo 2</p>
      <p>cubo 3</p>
</div>
<div>
      <p>cubo 4</p>
      <p class="cinco">cubo 5</p>
      <p>cubo 6</p>
</div>
<div>
      <p>cubo 7</p>
      <p>cubo 8</p>
      <p>cubo 9</p>
</div>
```



Propiedad Display. Recordamos y ampliamos

En HTML permite modificar la forma en que se muestra un elemento. Por ejemplo, en HTML los elementos de tipo p (párrafos) se entiende que se mostrarán como párrafos independientes, es decir, con cada etiqueta p comienza un nuevo párrafo o bloque. No irán seguidos dos elementos de párrafo consecutivos. Lo mismo ocurrirá con el elemento div.

Como ya vimos en las listas, gracias a display , el comportamiento de un elemento se puede modificar.

Principales valores de display

- none. El elemento queda oculto (también los elementos hijos) y el espacio que ocupaba el elemento se retira. Desaparecerá incluso del árbol de elementos del documento, lo que impedirá, por ejemplo, el acceso a este elemento desde JavaScript.
- Inline. Se muestran sin romper la línea actual. Se muestran dentro de la línea. Por tanto podemos colocar dos elementos inline seguidos. Los elementos *inline* ignoran los márgenes superior e inferior, pero sí reconocen los rellenos y márgenes izquierdo y derecho, así como los bordes y los colores e imágenes de relleno. Ejemplos de elementos que son por defecto de este tipo serían: span , small , strong , me , mark , abbr , kbd , etc.

Principales valores de display

- **block**. Se muestran en una caja rectangular. Rompen la línea actual y, por defecto, no admiten ningún otro elemento a su lado (independientemente del tamaño o la alineación). Los elementos de tipo block admiten manipular cualquier propiedad. Elementos, por defecto, de este tipo son: **p**, **h1**, **div**, **section**, **artículo**, **nav**, **figure**, **ul**, etc.
- **inline-block**. El elemento forma un bloque rectangular, pero es interior en el contenedor (no rompe la línea actual). Sí admiten elementos a su lado. Admiten manipular las propiedades **width** y **height** lo que demuestra su semejanza con el valor **block**. Ejemplos de elementos de este tipo son: **button**, **image**, **input, textarea, select** , etc.
- Otros valores como **flex, inline-flex, grid, inline-grid,**

Más información en https://www.w3schools.com/css/css_display_visibility.asp

Ejemplo

```
<div class="moderno">
<p>Lorem ipsum, dolor sit amet consectetur adipisicing elit. Hic fugit tempora dignissimos cupiditate dolore quasi impedit, aliquam commodi nemo exercitationem velit perferendis nulla enim voluptatum illum mollitia accusamus ad veritatis?</p></div>
<div class="moderno">
<p>Lorem, ipsum dolor sit amet consectetur adipisicing elit. Rem a excepturi distinctio dolorem laboriosam similique earum repellat alias totam consequuntur ab magni saepe eaque, debitis, doloribus iste temporibus eum minus?</p></div>
<div class="moderno">
<p>Lorem ipsum, dolor sit amet consectetur adipisicing elit. Hic fugit tempora dignissimos cupiditate dolore quasi impedit, aliquam commodi nemo exercitationem velit perferendis nulla enim voluptatum illum mollitia accusamus ad veritatis?</p></div>
```

```
.moderno{
    display: inline-block;
    vertical-align: top;
    width:30%; /*ancho*/
    padding: 5px; /*relleno*/
    border: 2px solid darkred;
    border-top-left-radius: 20px;
    border-bottom-right-radius: 20px;
    }
```

Lorem ipsum, dolor sit amet consectetur adipisicing elit. Hic fugit tempora dignissimos cupiditate dolore quasi impedit, aliquam commodi nemo exercitationem velit perferendis nulla enim voluptatum illum mollitia accusamus ad veritatis?

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Rem a excepturi distinctio dolorem laboriosam similique earum repellat alias totam consequuntur ab magni saepe eaque, debitis, doloribus iste temporibus eum minus?

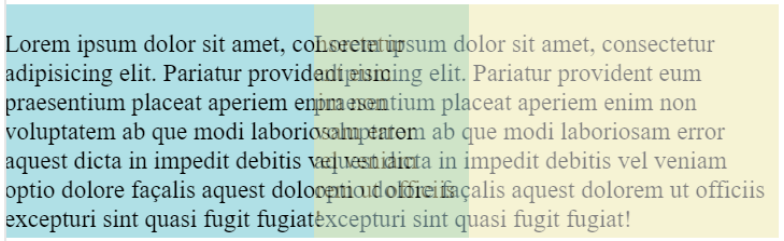
Lorem ipsum, dolor sit amet consectetur adipisicing elit. Hic fugit tempora dignissimos cupiditate dolore quasi impedit, aliquam commodi nemo exercitationem velit perferendis nulla enim voluptatum illum mollitia accusamus ad veritatis?

Opacity

- La opacidad consiste en que el elemento al que se aplique no se muestre al 100% de luz, sino que se muestra con un efecto de transparencia que permite ver lo que quedaría detrás suyo (que serán los elementos con menor índice z en la misma posición).
- Se controla con la propiedad **opacity** a la que se indica un número decimal entre **0** (transparencia total) y **1** (opacidad total).

Opacity Ejemplo

```css
div{
    position: fixed;
    width: 300px;
    height: 150px;
    top: 0;
}
.capa1 {
    z-index: 1;
    background-color: powderblue;
    left: 0;
}
.capa2 {
    z-index: 2;
    background-color: palegoldenrod;
    left: 200px;
    opacity: .5;
}
```




```html
<body>
    <div class="capa1">
        <p> Lorem ipsum dolor sit amet, consectetur adipisicing
        elit. Pariatur provident eum praesentium placeat aperiem
        enim non voluptatem ab que modi laboriosam error aquest
        dicta in impedit debitis vel veniam optio dolore façalis
        aquest dolorem ut officiis excepturi sint quasi fugit
        fugiat!</p>
    </div>
    <div class="capa2">
        <p> Lorem ipsum dolor sit amet, consectetur adipisicing
        elit. Pariatur provident eum praesentium placeat aperiem
        enim non voluptatem ab que modi laboriosam error aquest
        dicta in impedit debitis vel veniam optio dolore façalis
        aquest dolorem ut officiis excepturi sint quasi fugit
        fugiat!</p>
    </div>
</body>
```